

Organization: Buzzex

Application: Source Code Review | Buzzex

Source Code Review Assessment Report



Security is in Our DNA

1st Largest

Auditor

Market share (almost 100%)
of UPI assessment.

2nd Largest

Financial Institution

Of countries application
assessor.

3rd Largest

Telecom Company

End to end security
assessor.

Fortune 500

Oil and Gas company is
protected by us.

10 Billion Transactions

Assessed on more than
200+ banking applications.

Top Fortune 500

Companies vulnerabilities
have been managed by us.

Executive Summary

Background: Buzzex is a Web3-focused software development company that merges secure communication with decentralized finance (DeFi). Founded in 2024 by Syed Nasir and Shaikh Tahir, the Dubai-based company built its platform to challenge legacy apps like Telegram and Session.

Scope of Work: The objective of this exercise was to assess the Buzzex defense against unauthorized attacks and detect vulnerabilities that may pose a threat to its systems and the Application. The report also aims at providing recommendations and guidelines against such threats, which help in protecting the confidentiality and integrity of Applications. The testing process included the use of manual tests.

Approach and Methodology: The assessment was performed using methodologies based on industry-leading practices, such as the OWASP Code Review Guide, OWASP ASVS, NIST SP 800-115, SANS Top 25, and CERT Secure Coding Standards. The following steps were adopted as part of the methodology by the application security team:

- Methodology encompasses multiple stages, starting with information gathering to understand the application's architecture, programming languages used, frameworks, critical modules, and data flow patterns. This includes reviewing documentation, build processes, and project structure to identify security-sensitive components, APIs, and backend services.
- Configuration and dependency analysis involves reviewing configuration files, third-party libraries, and environment parameters for insecure settings, hardcoded secrets, outdated dependencies, and insecure integration points.
- Automated Static Application Security Testing (SAST) is conducted using industry-standard tools to detect common coding vulnerabilities such as injection flaws, insecure object deserialization, cross-site scripting, insecure cryptographic usage, insecure file handling, and improper input validation. The results from these tools are analyzed to eliminate false positives and identify high-risk issues.
- Manual code review is performed to detect complex vulnerabilities that automated tools may miss, such as business logic flaws, authorization bypasses, input validation gaps, privilege escalation paths, improper error handling, and unsafe data processing practices. This includes analyzing both client-side and server-side code, data flow, and trust boundaries to evaluate how sensitive data is handled throughout the application.
- Review of authentication and session management includes analyzing login flows, token handling, OTP validation, password hashing mechanisms, session expiration logic, and multi-factor authentication implementations. Authorization mechanisms are also reviewed to ensure proper enforcement of access rules based on user roles and permissions.
- Analysis of critical security controls includes evaluating input validation, output encoding, secure use of cryptographic functions, secure cookie attributes, exception handling, sensitive data exposure, API request/response security, and secure storage of keys, tokens, and secrets.
- Verification of coding practices based on secure coding standards, including CERT and OWASP guidelines, to ensure the application adheres to safe development practices. This involves reviewing the implementation of logging practices, error messages, data sanitization routines, and secure usage of APIs and system functions.
- All vulnerabilities identified during the assessment are mapped to industry-recognized standards such as CWE, OWASP Top 10, and SANS Top 25. Each issue is analyzed using the CVSS (Common Vulnerability Scoring System) to determine the impact and severity based on exploitability metrics and business risk.
- Document all findings, including code-level vulnerabilities, insecure configurations, and logical flaws. Assess the severity and impact of each finding and prioritize them based on risk. Provide actionable remediation steps, secure coding recommendations, and improvement guidelines to help the development team address the vulnerabilities effectively and ensure long-term security of the application.

Risk Classification

Critical Risk	These vulnerabilities can allow attackers to take complete control of your web applications and web servers. In exploiting this type of vulnerability, attackers could carry out a range of malicious acts.
High Risk	A High severity vulnerability, which means that on exploiting such vulnerabilities, attackers, can view information about your system that helps them find or exploit other vulnerabilities that enable them to access sensitive user and administrator information.
Medium Risk	Potential weakness in controls, which could develop into an exposure. Or issues that represent areas of concern and may impact controls. They should be addressed reasonably promptly.
Low Risk	Potential weaknesses in controls, which in combination with other weaknesses can develop into exposure. Suggested improvements not immediately/directly affecting controls.
Info Risk	Weaknesses mentioned under these sections are informational and are best practices. Either these weaknesses cannot be exploited directly or are very difficult to exploit due to multiple constraints.

ESOF Cyber Score Classification



Target

- Title: [Source Code Review](#) | [Buzzex](#)

Document Details

Document Type	Source Code Review Assessment Report
Client	Buzzex
Prepared	TAC Infosec Ltd.
Document Version	1.2
Type of Audit Report	Closure Report
Testing Type	White Box
Creation Date	21-09-2026

Document Version	Audit Period
1.0	20-08-2026 TO 25-08-2026
1.1	16-09-2026 TO 18-09-2026
1.2	19-09-2026 TO 21-09-2026

Author	Contact
Manav Kumar	manav.kumar@tacsecurity.com

Reviewed By	Contact
Yatiraj Poojari	yatiraj.poojari@tacsecurity.com

Observation Summary

Sr. No.	Vulnerabilities	Severity	Status
1	Hardcoded Passwords	High	Closed
2	Application is Vulnerable to SQL Injection	High	Closed
3	Hardcoded Keys	Medium	Closed
4	Application is vulnerable to XSS	Medium	Closed
5	PII (Personally identifiable Information) Information Disclosure	Medium	Closed
6	Debugging enabled	Low	Closed
7	Insecure CORS configuration	Low	Closed
8	Secure flag set to false	Low	Closed
9	Verbose Logging Enabled	Low	Closed
10	Improper implementation of CSRF token	Low	Closed
11	Insecure Local Storage of Credentials	Low	Closed
12	Use of Vulnerable and Outdated Components	Low	Closed

1

Hardcoded Passwords

Severity Level:	High
Status:	Closed
CWE ID:	798
CVSS3 Score:	7.5

Classifications:
A06:2025 Insecure Design

Method of Identification and Validation:
Manual

Background:
Password management refers to the practices and procedures implemented to securely handle and store passwords within a system or application. One common security issue related to password management is the presence of hardcoded passwords. Hardcoded passwords are passwords that are directly embedded in the source code of an application, script, or configuration file, instead of being stored securely or entered by the user during runtime.

Business Impact:
The impact of hardcoded passwords in password management is highly detrimental to the security of an application or system. Hardcoded passwords refer to the practice of embedding passwords directly within the source code or configuration files of software applications.

Revalidated Proof of concept :
As seen in the screenshot below, hardcoded password has been removed.

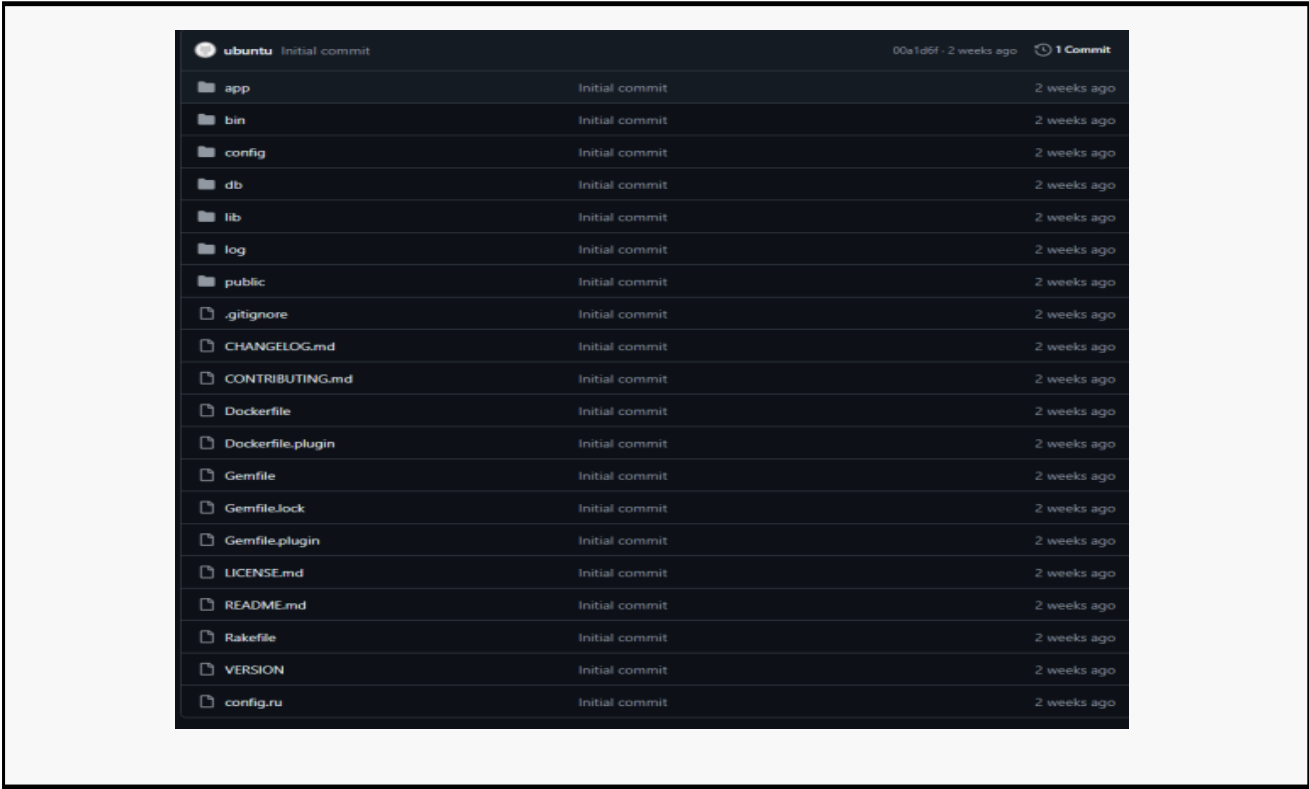


Figure 1. 1

Remedies:
If you need to store passwords in configuration files or databases, consider using encryption or hashing to protect them. Only the encrypted or hashed values should be stored, and the application should decrypt or validate the passwords when necessary.

2 Application is Vulnerable to SQL Injection

Severity
Level:

High

Status:

Closed

CWE ID:

22

CVSS3 Score:

7.4

Classifications:

A05:2025 - Injection

Method of Identification and Validation:

Manual

Background:

A SQL injection attack consists of insertion or "injection" of a SQL query via the input data from the client to the application. A successful SQL injection exploit can read sensitive data from the database, modify database data (Insert/Update/Delete), execute administration operations on the database (such as shutdown the DBMS), recover the content of a given file present on the DBMS file system and in some cases issue commands to the operating system.

Business Impact:

SQL injection attacks allow attackers to spoof identity, tamper with existing data, cause repudiation issues such as voiding transactions or changing balances, allow the complete disclosure of all data on the system, destroy the data or make it otherwise unavailable, and become administrators of the database server.

Revalidated Proof of concept :

As seen in the screenshot below, code uses parameterized ActiveRecord queries and an allowlist for dynamic ordering, with no direct SQL interpolation.

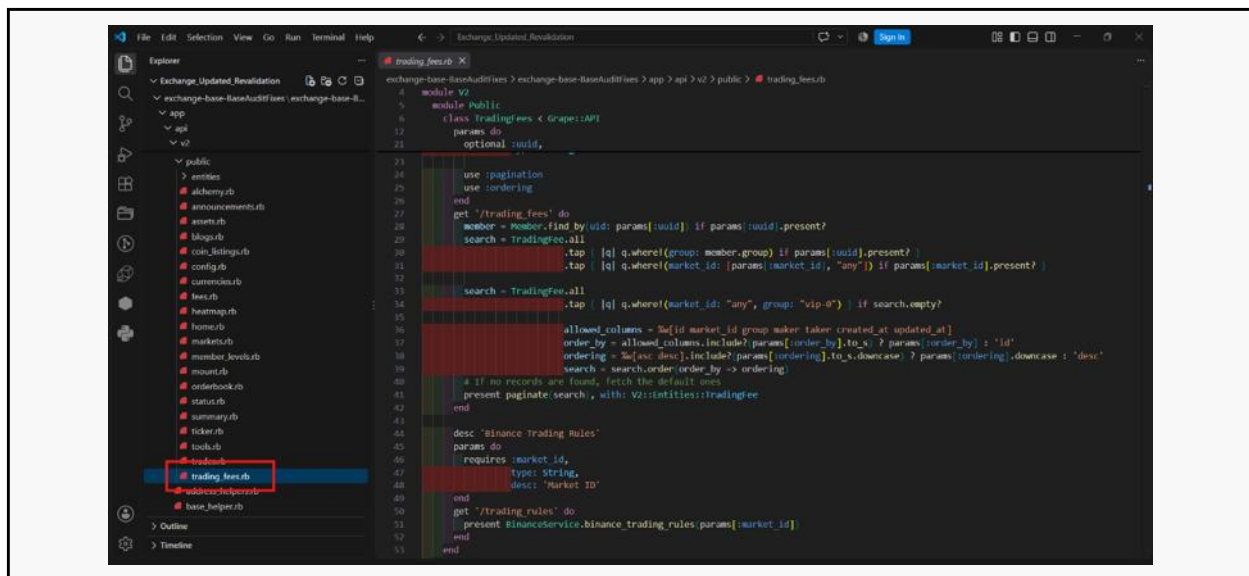


Figure 2. 1

Remedies:

- Assume all user-submitted data is evil so use input validation via a function such as MySQL's `mysql_real_escape_string()` to ensure that any dangerous characters such as `'` are not passed to a SQL query in data.
- You should also sanitize everything by filtering user data by context.
- For example, email addresses should be filtered to allow only the characters allowed in an e-mail address, phone numbers should be filtered to allow only the digits allowed in a phone number, and so on.
- Don't use dynamic SQL – don't construct queries with user input: Even data sanitization routines can be flawed, so use prepared statements, parameterized queries or stored procedures instead whenever possible. But don't forget that while stored procedures prevent some types of SQL injection attacks, they fail to protect against many others, so don't rely exclusively on their use for your security

Medium

Closed

798

5.5

Manual

Hardcoding keys directly into your source code poses security risks because anyone with access to the code can potentially view and abuse the key. If the code repository is public or accessible to unauthorized users, the key can be compromised.

Hardcoding keys exposes them directly in the source code. If an attacker gains unauthorized access to the source code repository or the deployed application binaries, they can easily extract the API keys. This can lead to unauthorized access to sensitive data or services, potentially resulting in data breaches or service abuse.

As seen in the screenshot below, hardcoded keys have been removed.

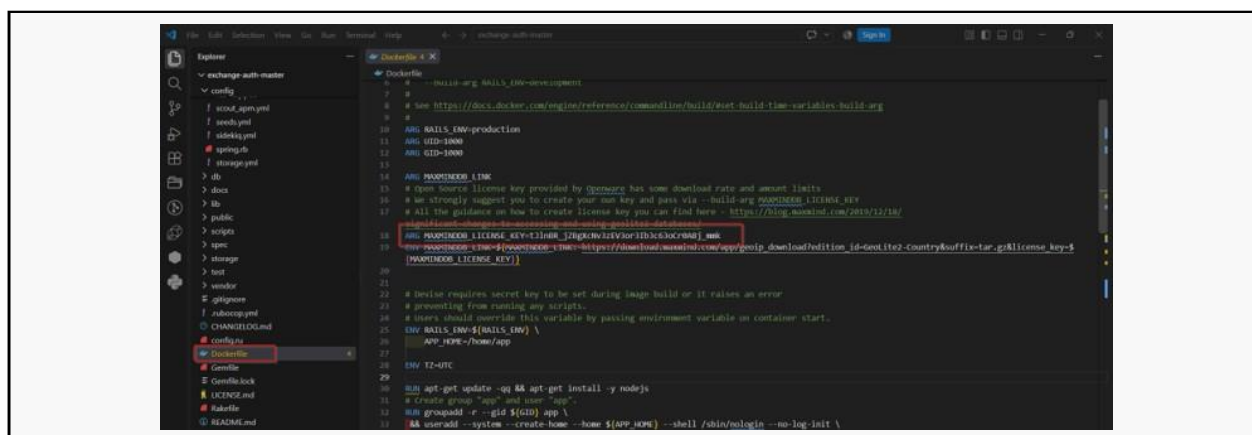


Figure 3. 1

Environment Variables: Store keys as environment variables on the system where your application is deployed. Your application can then read the key from the environment at runtime. Configuration Files: Store keys in configuration files that are not tracked by version control systems. These files should be kept secure and only accessible to authorized users. Secret Management Systems: Use secret management systems or vaults to securely store and manage keys. These systems provide encryption, access control, and auditing features for managing sensitive information.

4 Application is vulnerable to XSS

Severity Level:	Medium
Status:	Closed
CWE ID:	79
CVSS3 Score:	5.4

Classifications:

A05:2025 - Injection

Method of Identification and Validation:

Manual

Background:

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user. Flaws that allow these attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates without validating or encoding it.

Business Impact:

An attacker can use XSS to send a malicious script to an unsuspecting user. The end users browser has no way to know that the script should not be trusted, and will execute the script. Because it thinks the script came from a trusted source, the malicious script can access any cookies, session tokens, or other sensitive information retained by the browser and used with that site. These scripts can even rewrite the content of the HTML page

Revalidated Proof of concept :

As seen in the screenshot below, the dangerouslySetInnerHTML implementation is commented out and not executed.

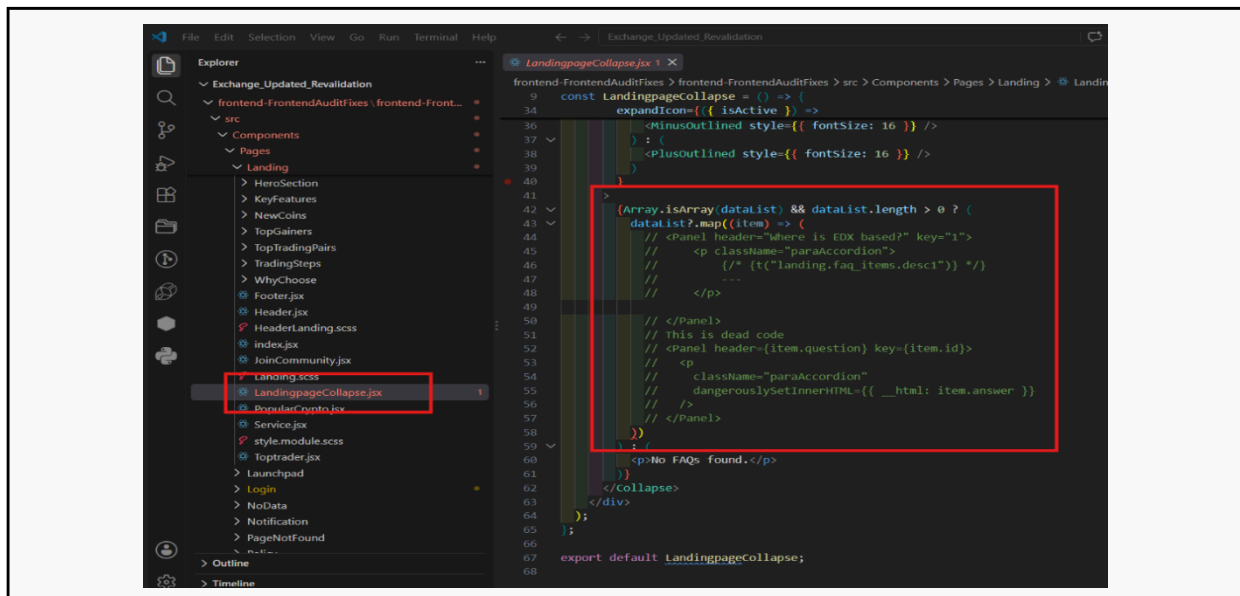


Figure 4. 1

Remedies:

Patch throughout the Application. 1. Create a whitelist of characters required by the Application. Once this whitelist is ready, the Application should disallow all requests containing any character apart from those in the whitelist. 2. Never insert untrusted data except in allowed locations. 3. HTML escape before inserting untrusted data into HTML element content. 4. JavaScript escape before inserting untrusted data into JavaScript data values. 5. CSS Escape and Strictly Validate before inserting untrusted data into HTML Style property values. 6. URL Escape before inserting untrusted data into HTML URL parameter values.

5 PII (Personally identifiable Information) Information Disclosure

Severity Level:

Medium

Status:

Closed

CWE ID:

312

CVSS3 Score:

5.4

Classifications:

A04:2025 - Cryptographic Failures

Method of Identification and Validation:

Manual

Background:

PII exposure refers to the unauthorized sharing or access of Personal Identifiable Information (PII). Cyber attacks, data breaches, human error, and unauthorized access and sharing by Data Brokers and People Search sites can all result in PII exposure.

Business Impact:

Information disclosure vulnerabilities can have both a direct and indirect impact depending on the purpose of the website and, therefore, what information an attacker is able to obtain. In some cases, the act of disclosing sensitive information alone can have a high impact on the affected parties.

Revalidated Proof of concept :

As seen in the screenshot below, Personally Identifiable Information file has been removed.

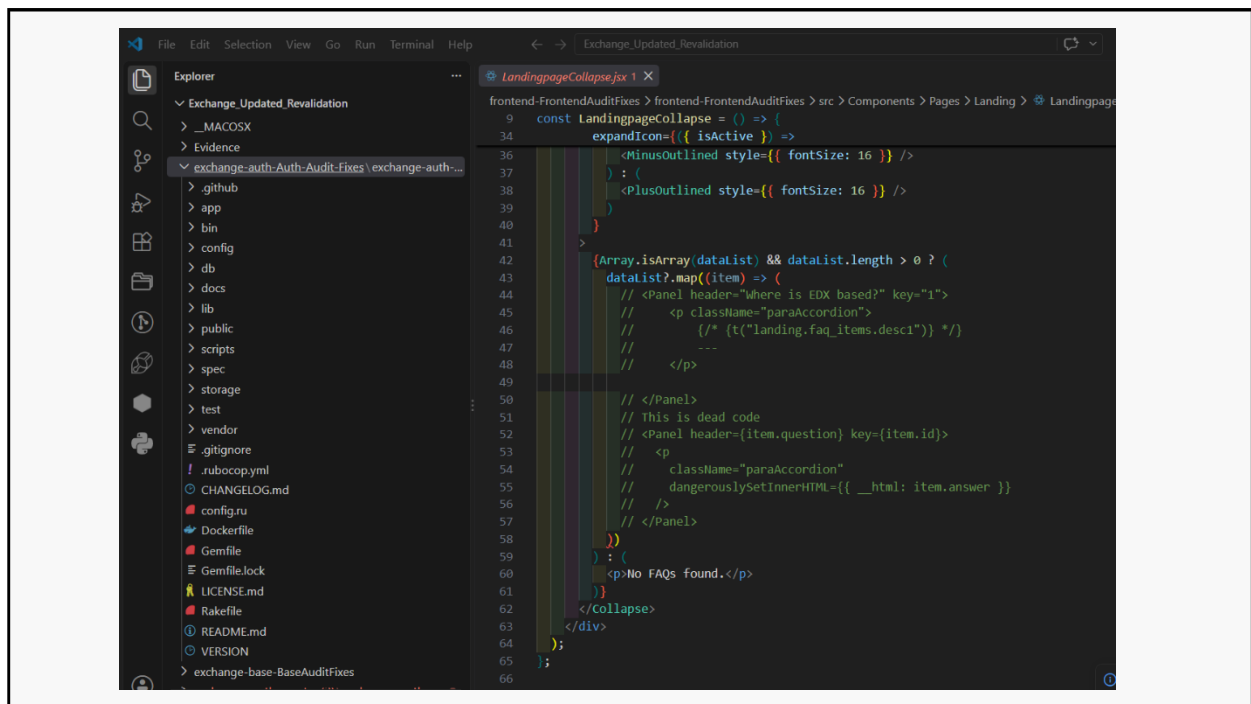


Figure 5. 1

Remedies:

Preventing information disclosure completely is tricky due to the huge variety of ways in which it can occur. However, there are some general best practices that you can follow to minimize the risk of these kinds of vulnerability creeping into your own websites. Make sure that everyone involved in producing the website is fully aware of what information is considered sensitive. Sometimes seemingly harmless information can be much more useful to an attacker than people realize. Highlighting these dangers can help make sure that sensitive information is handled more securely in general by your organization. Audit any code for potential information disclosure as part of your QA or build processes. It should be relatively easy to automate some of the associated tasks, such as stripping developer comments. Use generic error messages as much as possible.

6 Debugging enabled

Severity
Level:

Low

Status:

Closed

CWE ID:

489

CVSS3 Score:

3.1

Classifications:

A02:2025 Security Misconfiguration

Method of Identification and Validation:

Manual

Background:

The system has the debug mode active which generates an information leak when an error is generated.

Business Impact:

Obtain sensitive information such as stacktraces and versions of the systems used.

Revalidated Proof of concept :

As seen in the screenshot below, debugging has been disabled.

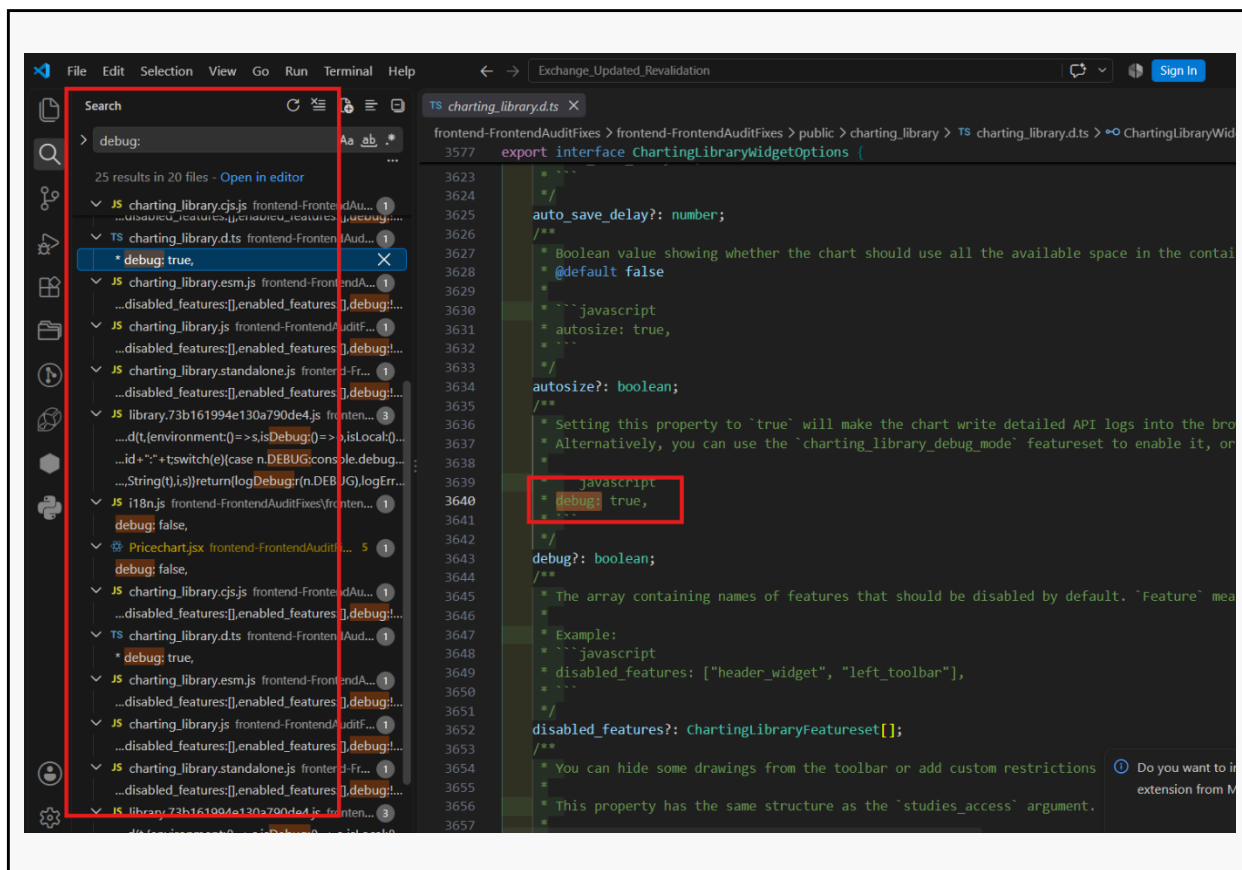


Figure 6. 1

Remedies:

Make sure that debugging mode is not enabled in the production environment and remove statements or portions of code that are executing in debugging mode.

Low

Closed

942

3.1

A02:2025 Security Misconfiguration

Manual

An HTML5 cross-origin resource sharing (CORS) policy controls whether and how content running on other domains can perform two-way interaction with the domain that publishes the policy. The policy is fine-grained and can apply access controls per-request based on the URL and other features of the request. Trusting arbitrary origins effectively disables the same-origin policy, allowing two-way interaction by third-party Web sites. Unless the response consists only of unprotected public content, this policy is likely to present a security risk.

This is generally not appropriate when using the same-origin security policy. The only case where this is appropriate when using the same-origin policy is when a page or API response is consider completely public content and it is intended to be accessible to everyone.

As seen in the screenshot below, wildcard origin has been removed.

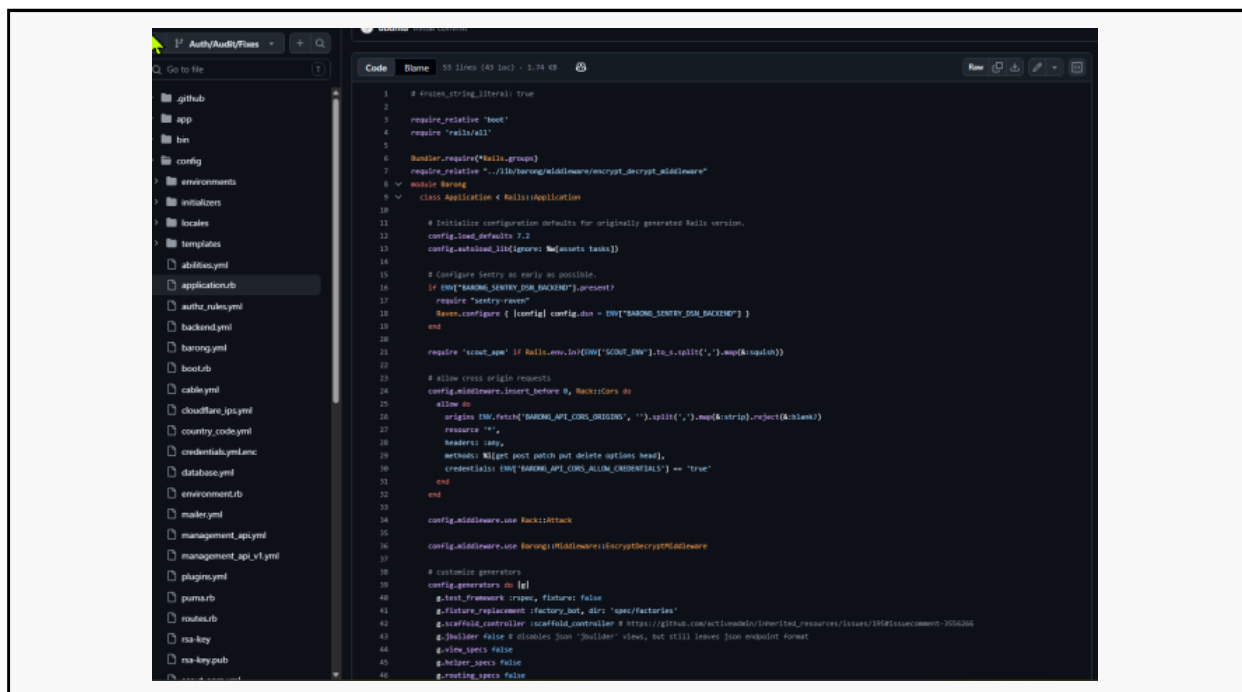


Figure 7. 1

Allow only selected, trusted domains in the Access-Control-Allow-Origin header.

8 Secure flag set to false

Severity
Level:

Low

Status:

Closed

CWE ID:

319

CVSS3 Score:

3.4

Classifications:

A04:2025 Cryptographic Failures

Method of Identification and Validation:

Manual

Background:

The configuration explicitly sets the application scheme to http and disables FORCE_SECURE_CONNECTION. As a result, the application may accept or serve requests over an unencrypted HTTP connection rather than forcing clients to use HTTPS. This can expose sensitive information transmitted between the client and server to network-level interception.

Business Impact:

An attacker positioned on the network could potentially intercept or manipulate HTTP traffic through a Man-in-the-Middle (MitM) attack. Depending on the data transmitted, this could lead to exposure of authentication credentials, session cookies, API tokens, personal information, or transaction data, as well as traffic modification.

Revalidated Proof of concept :

As seen in the screenshot below, the application forces all access over SSL/TLS when FORCE_SECURE_CONNECTION is set to true.

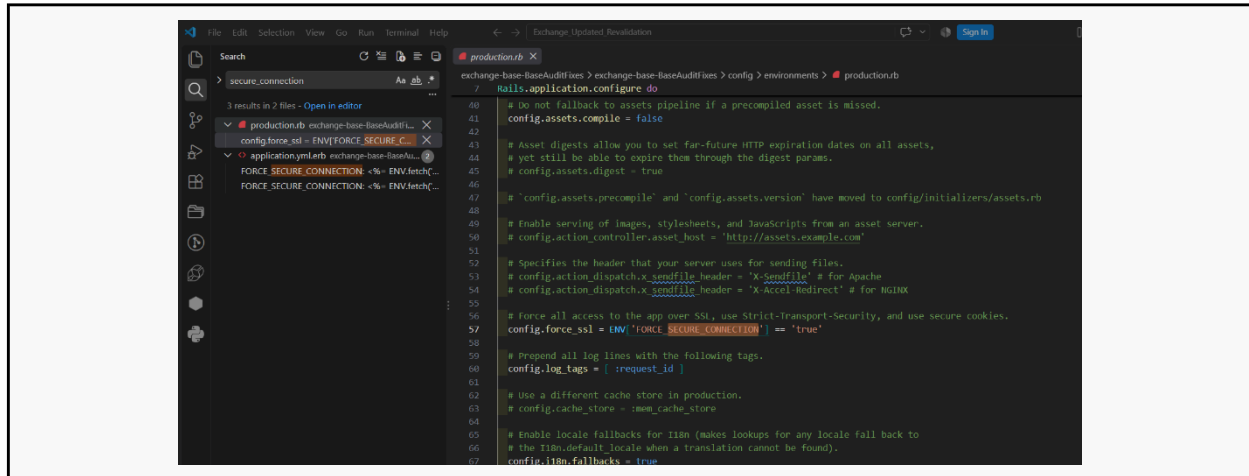


Figure 8. 1

Remedies:

Enforce HTTPS instead of allowing HTTP, restrict CORS to trusted origins instead of using a wildcard, and make sure sensitive services like databases or caches aren't exposed without authentication.

9 Verbose Logging Enabled

Severity
Level:

Low

Status:

Closed

CWE ID:

532

CVSS3 Score:

3.1

Classifications:

A02:2025 - Security Misconfiguration

Method of Identification and Validation:

Manual

Background:

Debug logging is generally intended for development and troubleshooting. Enabling it in a production environment can cause excessive internal application information to be recorded and potentially exposed through logs.

Business Impact:

Debug-level logging may disclose sensitive technical information such as application logic, internal paths, database queries, authentication/session details, request parameters, stack traces, and other implementation details. If logs are accessible to unauthorized users, this information could assist attackers in reconnaissance and exploitation of additional vulnerabilities.

Revalidated Proof of concept :

As seen in the screenshot below, debug logging is not enabled.

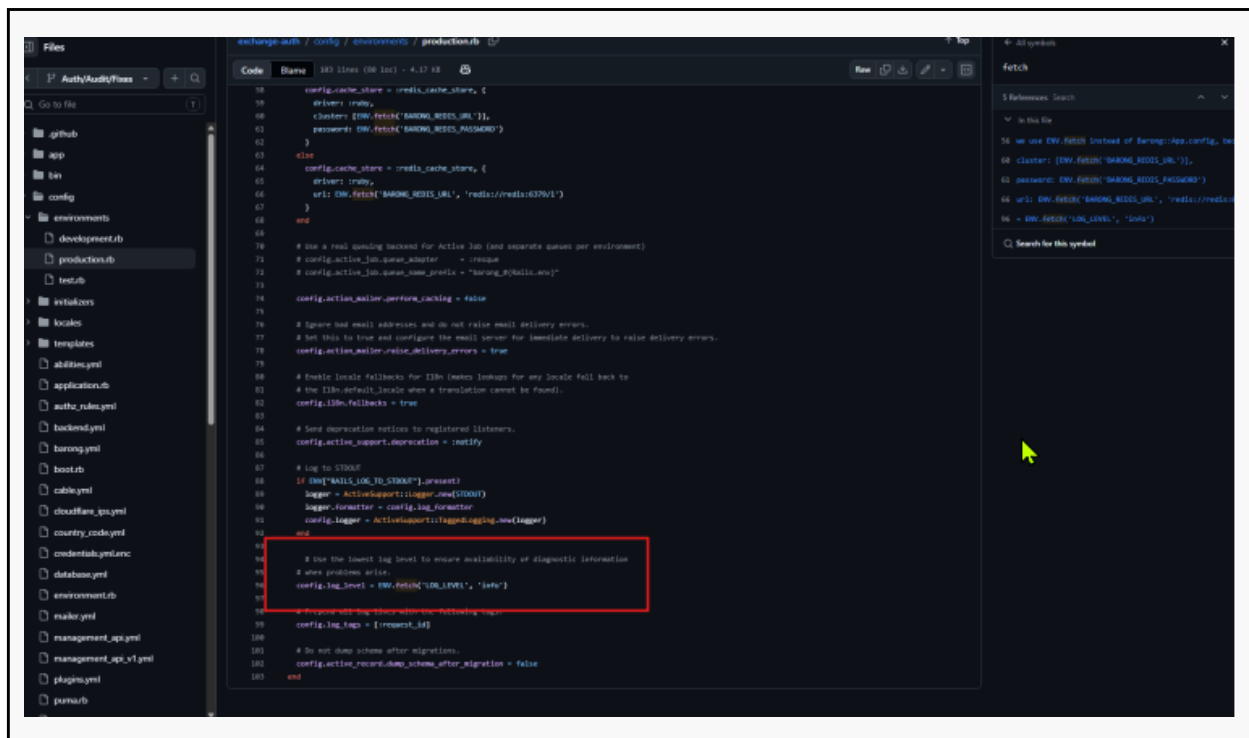


Figure 9. 1

Remedies:

Change the production default logging level from debug to info or another appropriate production level. Ensure debug logging is enabled only in controlled development/testing environments.

10 Improper implementation of CSRF token

Severity
Level:

Low

Status:

Closed

CWE ID:

79

CVSS3 Score:

3.1

Method of Identification and Validation:

Manual

Background:

LocalStorage is accessible to JavaScript running under the same origin, the CSRF token is not protected from malicious scripts executing in the application's origin. In the event of an XSS vulnerability, an attacker could read the stored token and potentially use it to bypass CSRF protections for requests requiring that token.

Business Impact:

If an attacker can execute JavaScript in the application's origin, they may be able to retrieve the CSRF token from localStorage and use it in forged requests. This can weaken the application's CSRF protection and increase the impact of an existing XSS vulnerability.

Revalidated Proof of concept :

As seen in the screenshot below, CSRF protection is implemented using an XSRF-TOKEN cookie and corresponding X-XSRF-TOKEN request header, with the token no longer stored in browser localStorage.

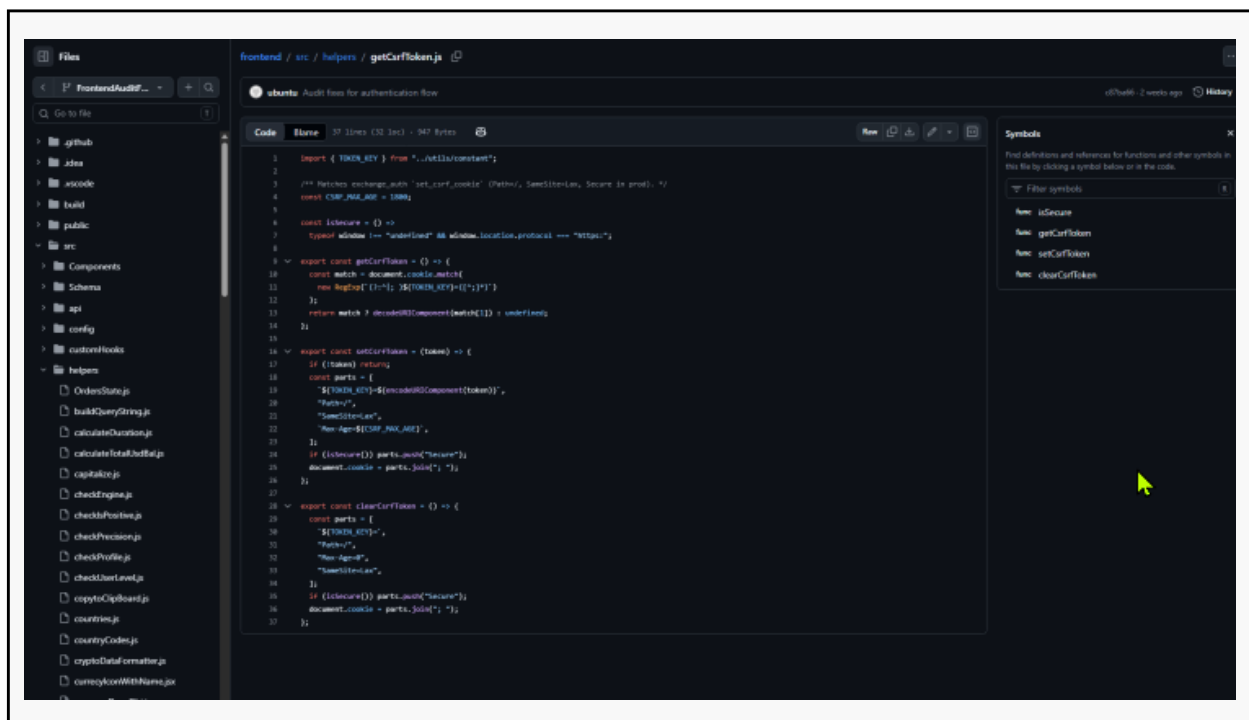


Figure 10. 1

Remedies:

It is recommended to implement strict token validation thorough validation of received CSRF tokens to ensure their authenticity and integrity before processing any users requests. Use a CSRF cookie with appropriate SameSite protection where applicable. Use a synchronizer-token pattern where the token is maintained server-side. If a client-readable CSRF token is required, ensure it is scoped appropriately and has a short lifetime/rotation strategy.

11 Insecure Local Storage of Credentials

Severity
Level:

Low

Status:

Closed

CWE ID:

922

CVSS3 Score:

3.1

Classifications:

A06:2025 Insecure Design

Method of Identification and Validation:

Manual

Background:

The CreateAccount.jsx component collects registration data and continuously saves the form values to browser localStorage whenever they change. The stored object explicitly includes password and confirmPassword fields before being serialized with JSON.stringify().

Business Impact:

If an attacker gains JavaScript execution in the application's origin, such as through XSS, they can access localStorage and retrieve the stored registration information, including the user's password and other personal data. Browser extensions or compromised client environments may also potentially access this data.

Revalidated Proof of concept :

As seen in the screenshot below, the sensitive data storage logic is commented out and is not active at runtime.

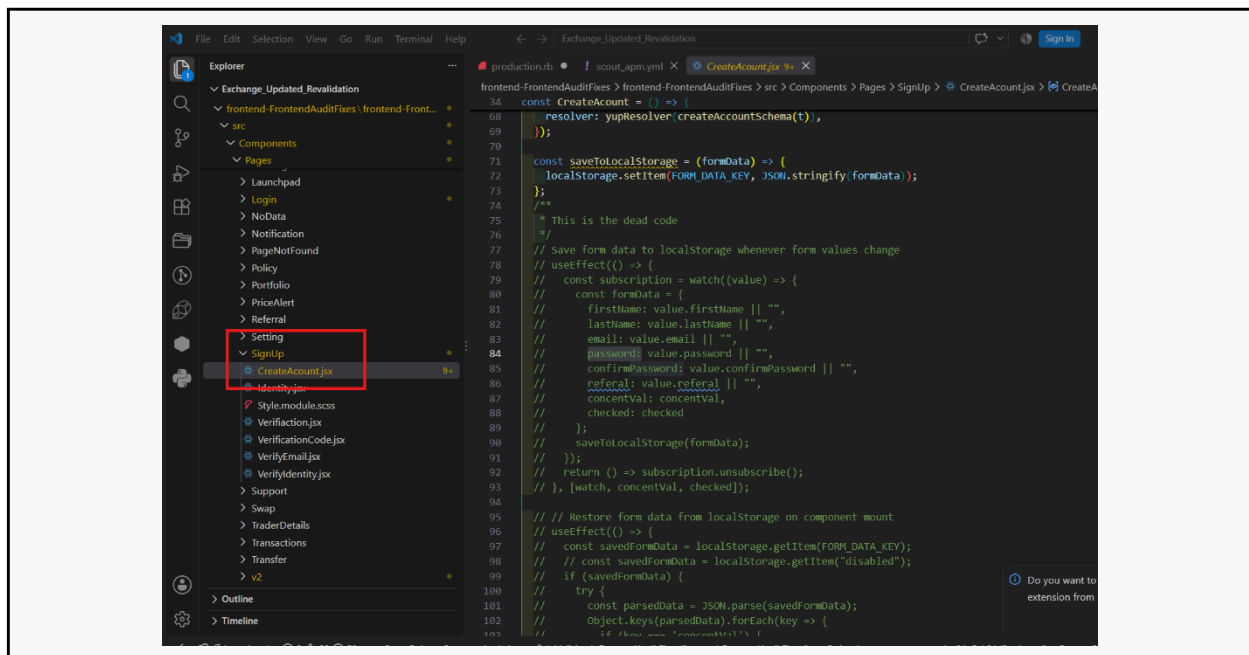


Figure 11. 1

Remedies:

It is recommended to avoid persisting credential information on the client system. Never store passwords or confirmation passwords in localStorage. Store only non-sensitive form fields that genuinely need persistence. Remove password and confirmPassword from formData before calling saveToLocalStorage(). Clear any previously stored sensitive registration data after successful registration or when it is no longer required.

12 Use of Vulnerable and Outdated Components

Severity
Level:

Low

Status:

Closed

CWE ID:

937

CVSS3 Score:

3.1

Classifications:

A03:2025 Software Supply Chain Failures

Method of Identification and Validation:

Manual

Background:

The application relies on vulnerable and outdated third-party components that contain known security flaws. These components are no longer fully supported or patched, making them attractive targets for attackers.

Business Impact:

Exploitation may lead to unauthorized access, data leakage, remote code execution, service disruption, and complete system compromise.

Revalidated Proof of concept :

As seen in the screenshot below, application is using axios version 1.20.0 which is safe to use.

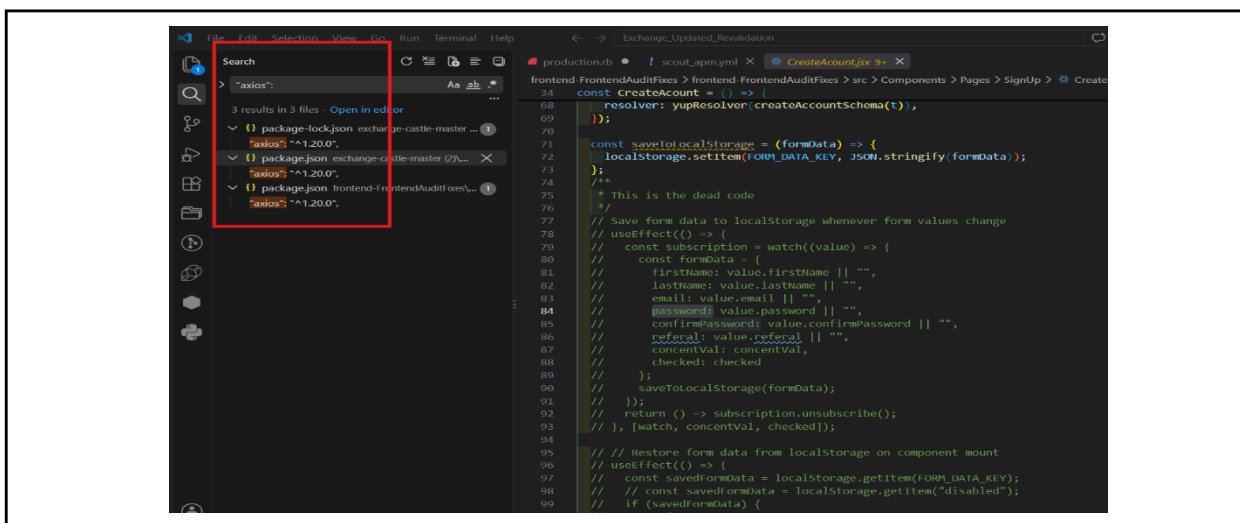


Figure 12. 1

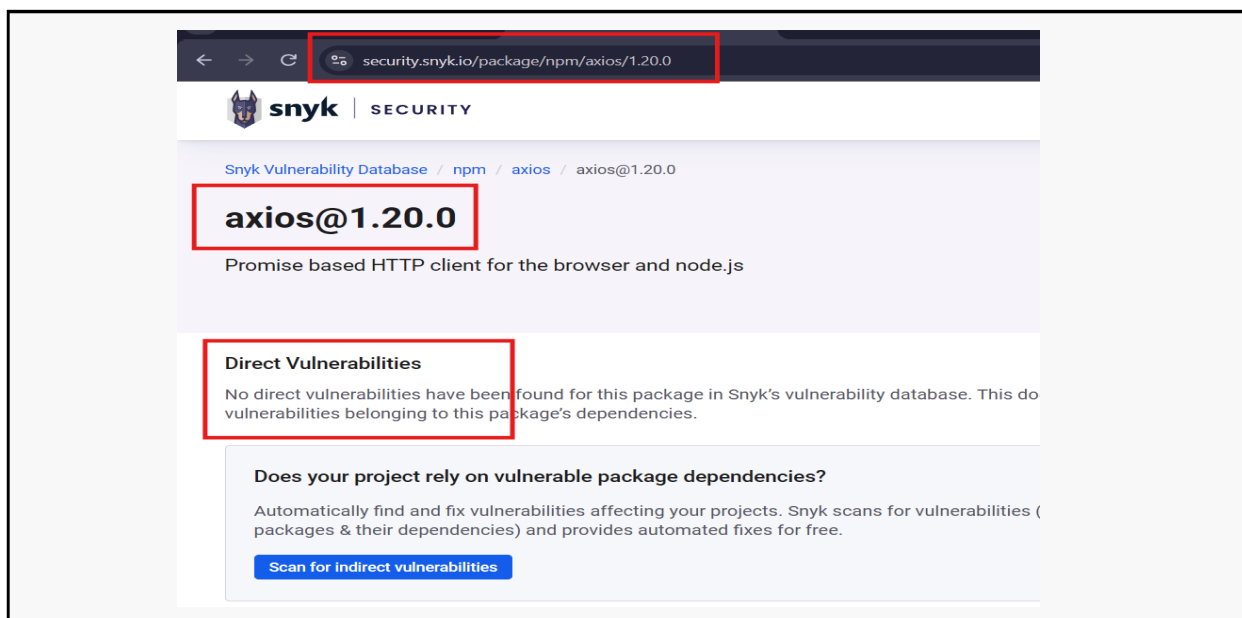


Figure 12. 2

Remedies:

Maintain an inventory of dependencies, update components regularly, remove unused libraries, and implement continuous vulnerability monitoring and patch management.



mail@tacsecurity.com | tacsecurity.com

© TAC Security. All rights reserved. This document is copyright protected. No information contained herein, shall, for any purpose other than its intended purpose, be disclosed, transmitted, duplicated, and used in whole and/or in part without prior written permission of TAC Security. Any redistribution or reproduction of part or all of the contents in any form is prohibited and will tantamount to a breach. TAC Security Solutions does not claim this report to be error free, even though every care has been taken to prepare the same.

Copyright © TAC Security - All Rights Reserved.